

Process Synchronization / Synchronization Tools

☐ Process Synchronization is a concept in operating system that ensures safely sharing of data and resources without errors and inconsistency.

☐ When two process can modify or influence each other, then they are called cooperating processes.

→ They must have a common resource or common memory, which they will use.

→ If ~~one~~ data changes but the other process doesn't notice, then there will be error.

Bounded buffer Problem / Producer-Consumer Problem

Created by process,

i) Producer Produce Process,

ii) Consumer Process,

* Buffer : → An array type memory that temporarily stores newly created process.

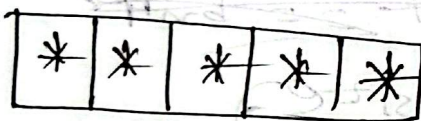
* Buffer Zone : → A neutral area in the middle of war zone.

* Bounded Buffer : → A fixed sized buffer memory.

* Bounded buffer Problem; contains 3 problems within,

- Buffer Overflow
- Buffer Underflow
- ~~Buffer~~ Race Condition

* Buffer Overflow; → Buffer is full



buffer-size = 5

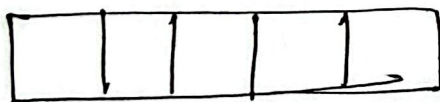
→ Producer creates an item.

Now, producer wants to keep another

* in it.

* Buffer Underflow; → Buffer ~~has~~ is empty

→ Consumer wants an item.



buffer-size = 5

* Race Condition; → Buffer is shared

→ Buffer becomes inconsistent,

→ If producer and consumer want to ^{manipulate} change the buffer at the same time and arise inconsistently.

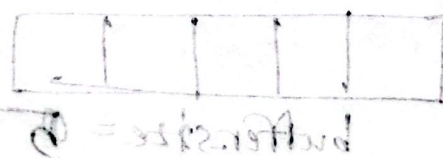
Count : → A variable that counts the number of variable in buffer.

while (true)

~~count = 0~~ ~~count = buffer size~~

(count == buffer size)
block person process

clock == 0
block consumer process.



Race Condition : → Buffer becomes inconsistent.

→ If producer and consumer want to change the value of the same-time, an error occurs. Inconsistency.

Producer P Consumer C

book →
pdf → 336

count++

count--

reg₁ = count

reg = count

reg₁ = reg + 1

reg₂ = reg - 1

Count = reg₁

~~count~~ count = reg₂

Critical Section

Instructions / code = process

However, not all of them are important

Important: instructions that manipulate shared resources

operation with shared memory location

This shared memory location is called

critical point

A vital section of a process that

can manipulate the shared item

Target: Secure & efficient execution

Critical Section Problem

□ Race Condition:

When multiple process shared the same data/item for their sequence of operation if the shared item becomes inconsistent, it is called race condition.

Basically shared resources has inconsistent value.

□ Critical Section:

Σ instructions/code = process

However, not all of them are important.

Important: ~~arith~~ arithmetic operation with shared memory location.

→ This shared memory location is called critical point.

CS: A vital section of a process that can manipulate the shared item.

Target: Secure & Efficient execution

Based on CS, the process is divided in a definite architecture.

Process i:

while (true)

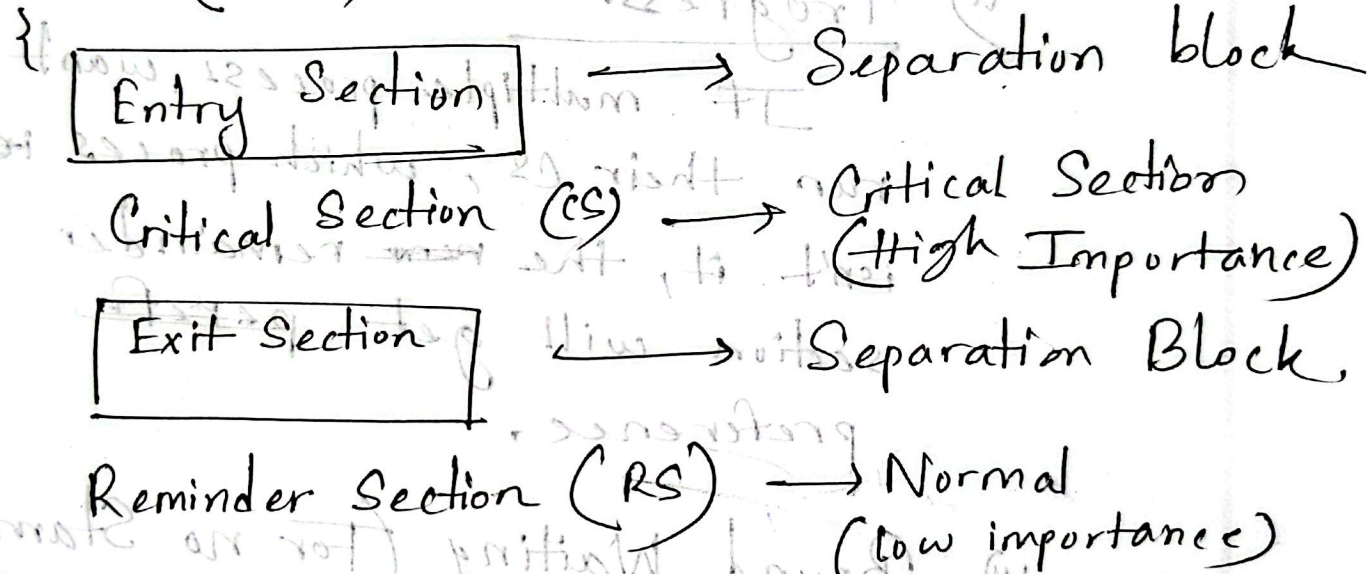


Fig: Physical Section Architecture

⇒ Condition :

If one process runs its CS, then other process can not run thier CS.
the CS

iii To solve a problem with CS, we need three requirements. They are _____

i) Mutual ~~Execution~~ exclusion.

ii) Progress.

If multiple process want to run their CS, which process ~~isn't~~ isn't it, the ~~rem~~ remainder section will get ~~perfer~~ preference.

iii) Bound Waiting (For no Starvation)

→ There will be a fixed number to enter a process CS.

05-05-26

☐ Solution of Critical Section Problem; ☐

The solutions can be broadly divided in three categories. They are →

i) Software based solution → Peterson's Solⁿ

ii) Hardware → Test And Set
→ Disabling Interrupt
(Uniprocessor)
→ Swapping

iii) High-level / Real-OS solution

→ Semaphore

→ Mutex Lock

→ Monitor

☐ Software Based:

Due to ~~dissimilar~~ difference

there is mismatch in Peterson's solution when applied to hardware.

It works two process,

□ Peterson's Solution:

boolean flag[2],
int turn;

while (true) {

flag[i] = true;

turn = i;

while (flag[j] && turn == j);

/* Critical Section */

flag[i] = false;

// Reminder section

}

~~while~~

while (true) {

flag[i] = true

turn = i

while (flag[j] && turn == j);

/* Critical Section */

flag[i] = false

// Reminder

}

/* Progress */

W.H.

// bounded Waiting.

③

- i) Use two global variable
- ii) flag is an array of size 2

iii) turn achieve { count }



The process if it ready for CS-

④ Bounded Waiting

H.W

i) Drawbacks of Peterson's Solution,

ii) Why it isn't used in modern computer OS synchronization?

Hardware Based Solⁿ:

* TestAndSet ()

```
boolean TestAndSet(*target)
```

```
{
```

```
    boolean rv = *target;
```

```
    *target = true;
```

```
    return rv;
```

```
}
```

Process

```
do {
```

```
    while (TestAndSet(&lock));
```

```
    /* Critical Section */
```

```
    lock = false;
```

```
    // Remainder Section
```

```
}
```

① [first checks condition]

(i) First check checks condition

(ii) Calls TestAndSet() function.
Block is a pointer. parameter reference

(iii) lock & reference is in target.

lock = true = CS is being used.

lock = false = CS is being executed.

lock = boolean value,

(iv) function

rv = lock & value

forcefully target & value true & CS from

rv & value true. So, lock & value & reference

(v) true value return while condition.
So infinity loop.

/*

run 1/2 2/2 3/2

Exam! What is Busy Waiting/Spin lock

①) मल्ल विन प्रकृति process ①

②) lock = false

lock = false

run करत तल lock false तल

प्रक. lock break तल

Get { target forcefully true कर तल मात
कार operation CS तल कर मल्ल
CS locked तल मल्ल -

① Test

if positive
set

2 that's why it
called test
and set

②

Semaphore: ^{positive}

A semaphore is an integer variable, apart from initialization, it is accessed through two atomic function $\text{wait}()$ and $\text{signal}()$.

$\downarrow$
P

$\downarrow$
V

atomic { The code written in $\text{wait}()$ and $\text{signal}()$ has to run altogether. There is no switching between function (like function call). That's why it is called atomic code. }

→ Invented by Dutch Computer Scientist.

<pre>wait(s) { while(s <= 0) s--; }</pre>	<pre>signal(s) { s++; }</pre>
---	-----------------------------------

Basic
Analogy

The key to enter a locked room = semaphore
If available:
enter
~~if~~ else:
wait

available | $S = \text{Semaphore}$
 $S = \text{value}$

S is allocated

wait is called : S value goes down by 1

that particular S can not be taken

when the resource is done

Signal is called : S value goes up.

☐ $S = 1$; lowest value : 0
highest " : 1

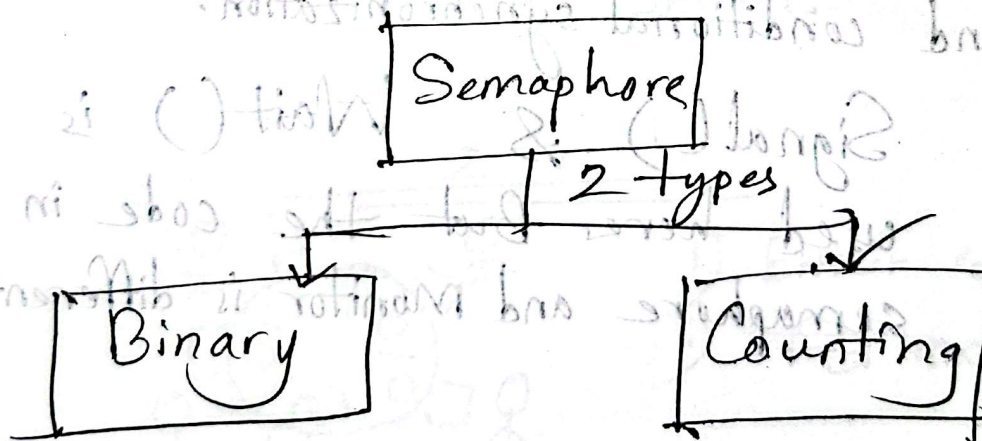
S_0 & S fluctuates between: $[0, 1]$

this is called Binary Semaphore

☐ $S = N$; lowest value : 0
highest " : N

fluctuates between $[0, N]$

this is called Counting Semaphore



H.W Qo How ~~Sem~~ Semaphore is used
to ~~rewrite~~ ~~solve~~ Produce & Consumer to
avoid race condition?

Rewrite the PSC problem
to avoid using Semaphore
to avoid Race Condition

☐ Monitor : requests are kept in a panel.
Only one gets the request
When done, the other gets.

ATM booth → Monitor

Money → Data / ~~Research~~ Resource

One man entry → Mutual exclusion

Functions → Critical Section

Out of money / refill → Conditional Synchronization

Def:

A monitor is a high-level synchronization abstract that combines mutual exclusion and conditional synchronization.

Signal() & Wait() is used here. But the code in semaphore and Monitor is different.

Problem - 1:

Reader-writer Problem:

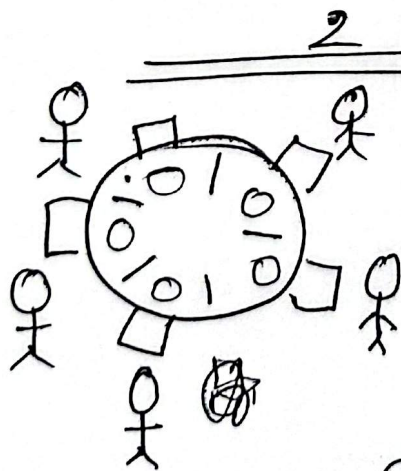
Problem - 2:

Dining Philosopher Problem:

Read() → allowed to multiple

write() → allowed only to one at a time

Read() : → Not allowed when the data is being written.



1) Either धात

2) छिन्न करव

किन्तु एकजसु काचि माजिरो 2

Worst Case - 1:

Starvation

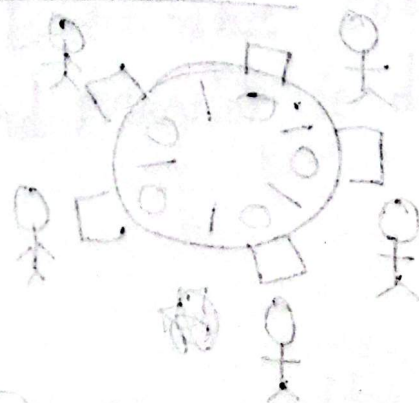
→ કમ્પ્યુટર મૂકી (ધાળો પાડે)
→ " " ઢિલા કરાડે

Worst Case - 2:

Deadlock

→ પ્રત્યેકે પ્રત્યે કલ્પે ઢાઢક મિલે
મિલે,

→ પ્રત્યેકે લોડે (ધાળો પાડે) ના



Memory Allocation: Two types.

→ Fixed Size

→ Variable Size

* Variable Size Memory Allocation:

→ First Fit : → Next Fit : a variation of first fit that start with previous point.

→ Best Fit

→ Worst Fit

First fit : First Come → Available size → Fit it

Best fit : First Come → Search all available size

↓
Put the Fit it where it takes the least space

Worst fit :

↓
Fit where it takes the most space

Chapter-9 : ^{Main Memory} Memory Management

☐ Two types of register:

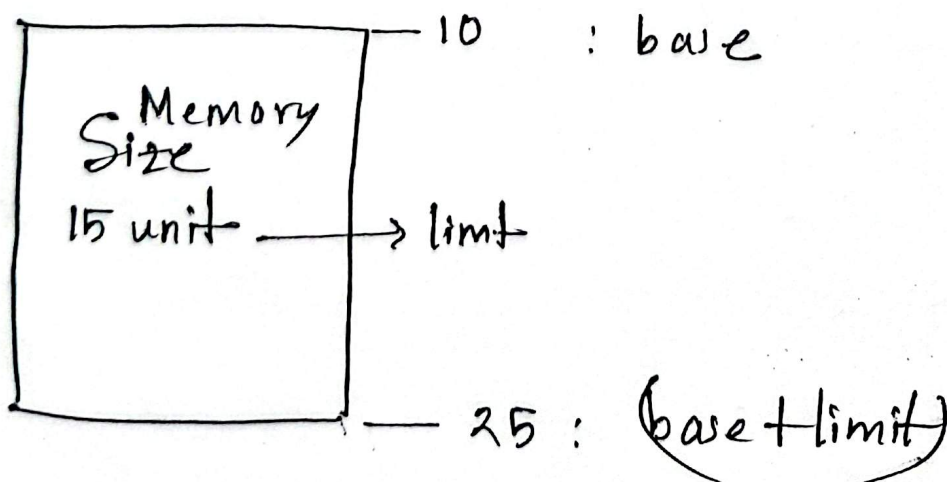
- Base register
 - Limit register
- } Initialized by OS

Base Register:

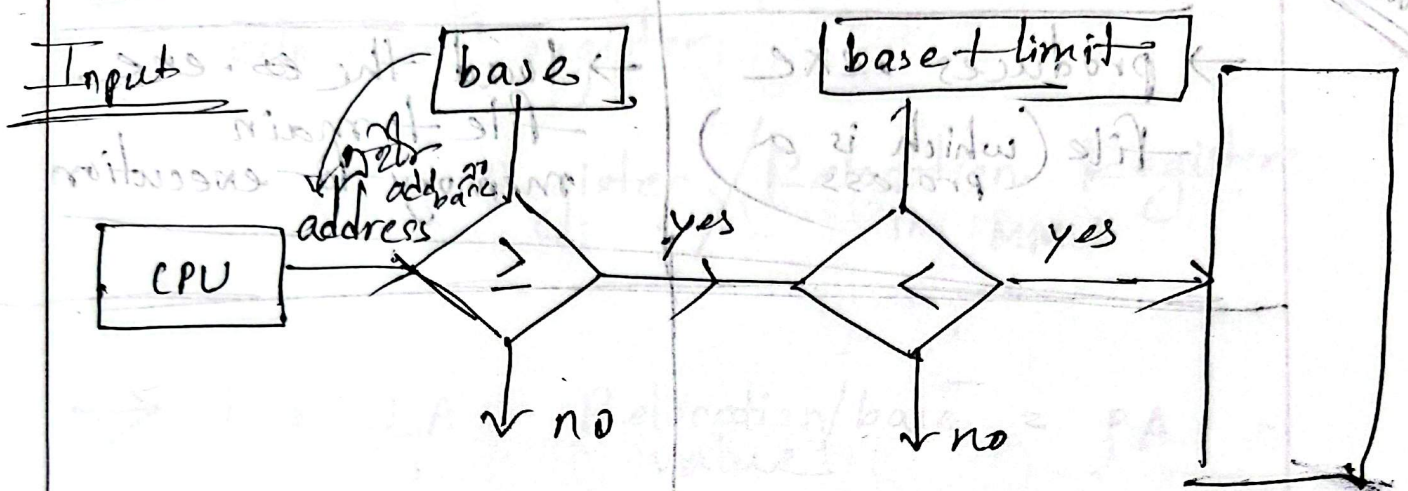
→ Starting address of a memory block.

Limit Register:

→ Size of each memory block.



Hardware Address Protection:

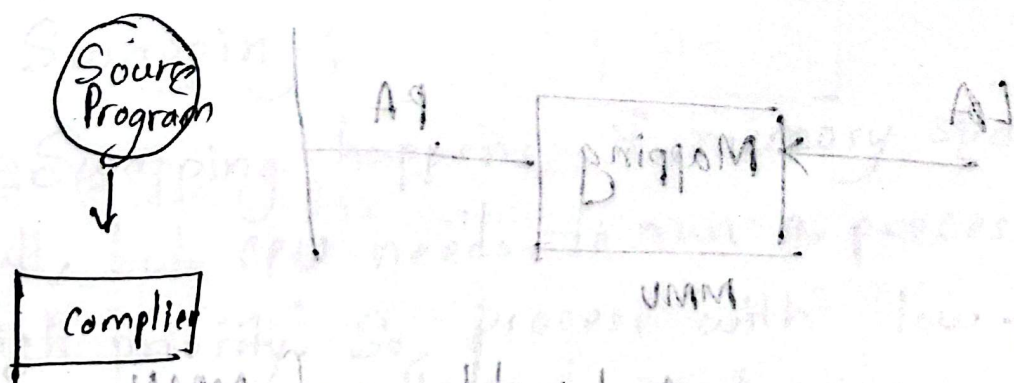


trap to operating system

Hardware address = PA (Physical Address) $\rightarrow 20 \text{ bit}$

Software calculated address = LA (Logical Address) $\rightarrow 16 \text{ bit}$

Multistep Processing of a User Program:



MMU to virtual address $\rightarrow$
 virtual address $\rightarrow$ physical address $\rightarrow$
 MMU to virtual address $\rightarrow$

Exam

linker

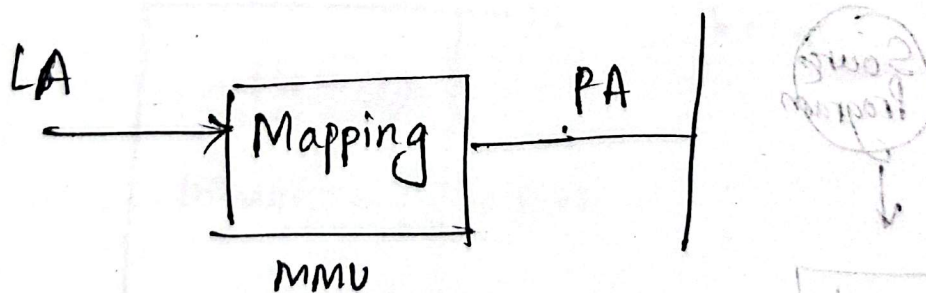
loader

→ produces .exe
file (which is a
process)

→ load the .exe
file to main
memory for execution

☐ Logical & Physical Address

MMU (Memory Management Unit)

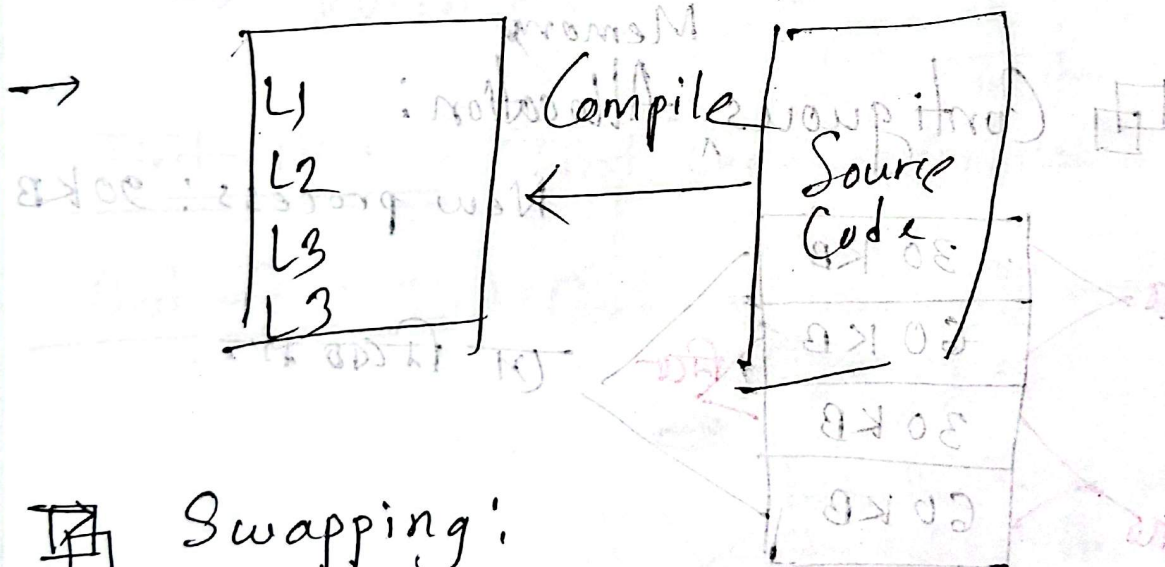


- Internal Architecture of MMU
- How this LA → PA conversion works?
- Working/Application of MMU

→ Base Register is known here as relocation register.

Base Register / Relocation Register
in MMU

→ $LA + \text{Relocation/base value} = PA$



Swapping:

Swapping happens if memory space is full, but CPU needs to run a process of high priority. So, process with low ~~priority~~ priority is ~~taken away~~ rolled out and temporarily stored elsewhere so the high priority process can be executed.

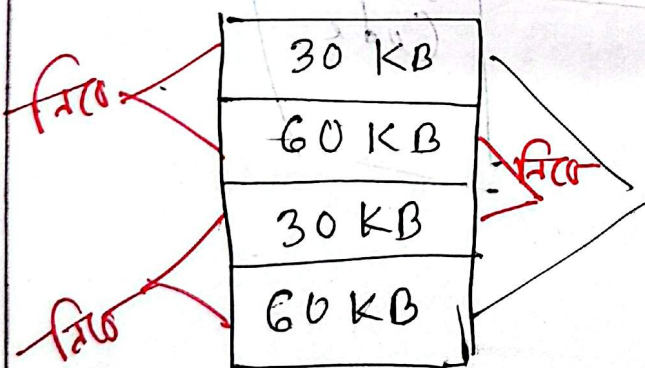
Roll in! High priority $\rightarrow$ Memory

Roll out: Memory $\rightarrow$ low priority

Temporary storage: Backstorage
for low priority

$\rightarrow$ OS get the highest priority in Memory

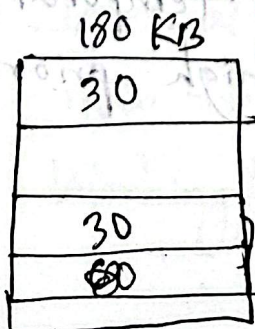
Memory
Contiguous Allocation:



New process: 90 KB

म रिक्त है

यदि memory space contiguous है,
तब memory space भरने पर process
place कर सकते हैं



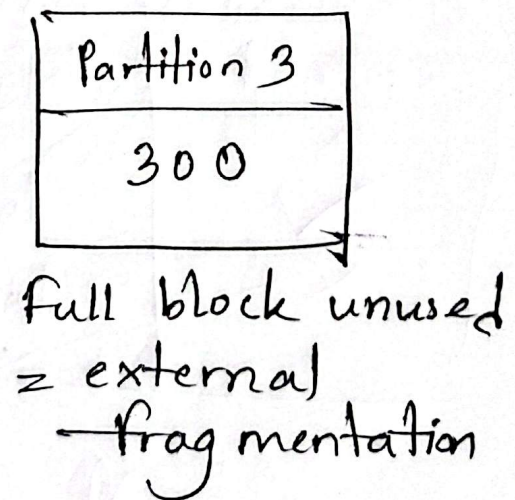
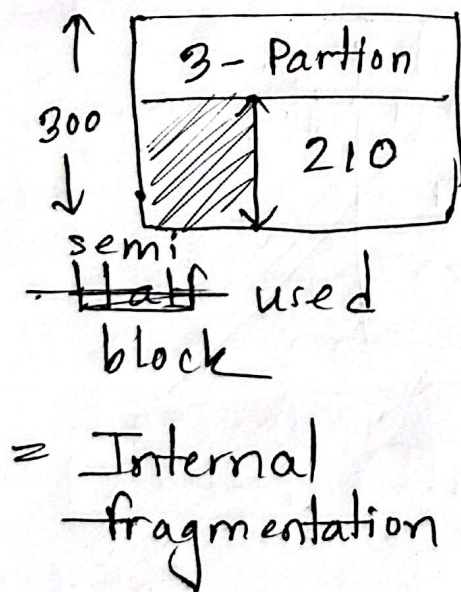
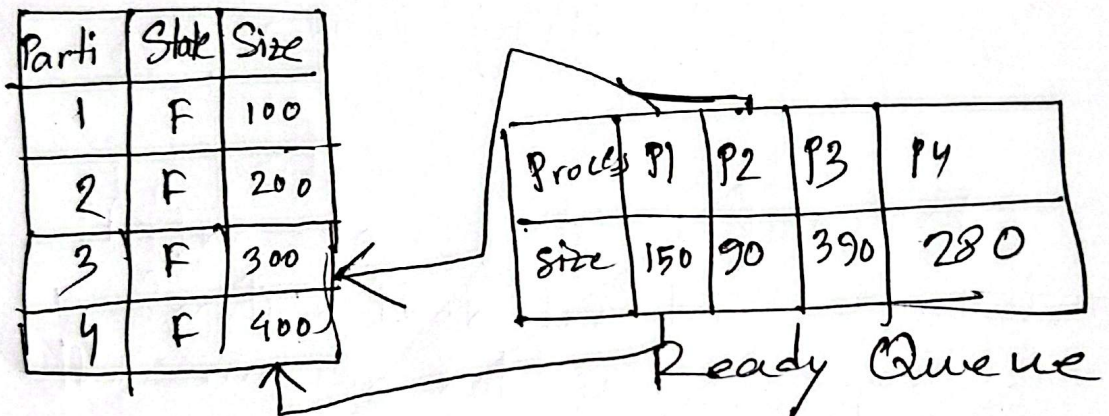
मरार = 120 KB / P = 90 KB

कि 120 contiguous है

So, 90 KB भर सकते हैं

Fragmentation

Fragmentation: Unused space in main memory.



Solution:

1) Compaction: Merge all free memory in one block which is large.

→ Possible if relocation is dynamic and is done at execution time.

2) Permit non-contiguous memory architecture

→ Paging: removes external fragmentation

→ Segmentation: internal

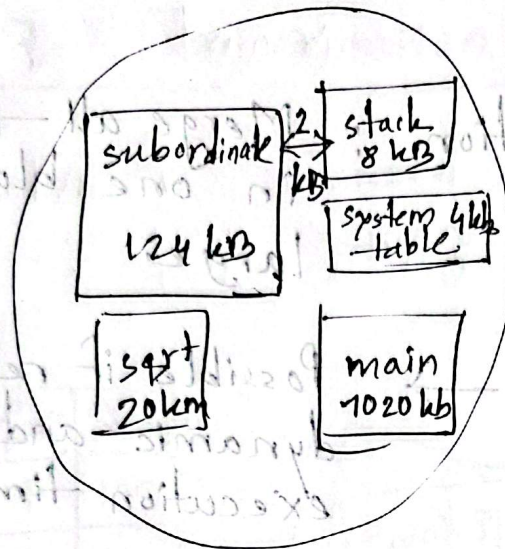
Segmentation

A program is a collection of segment.

যাৰ মন্ত্ৰীৰ দৰকাৰ (মন্ত্ৰীকৈ) জাহান
নিকো।

→ So, the segmentation size is of variable length.

→ Segmentation removes internal fragmentation completely.



If the space between two segment is smaller than the least segment size, then we can not use the space

This is called external fragmentation.

Segmentation	Fragmentation	remove status
	remove	
internal		✓
external		✗

☐ User Segment:

is identified using $\langle \text{segment_number}, \text{offset} \rangle$

segment_number: holds location.

offset: how much space is need for the segment.

Segment/index

Segment_number

Segment table



Physical Address

limit	base

Seg

base = the starting address of physical memory.

$$PA = \text{Base} + \text{Segment}$$

limit: physical memory

or block

maximum space.

Paging:

→ P.M is divided into ~~equal~~ equal size block named ~~the~~ frame

→ the equal ~~enpt~~ ~~enpt~~

→ Process logical address space ~~is~~ is divided into equal size block called pages.

~~2nd~~ page 3 KB ~~or~~ ~~1st~~ page 1 KB

page size = frame size

Logical

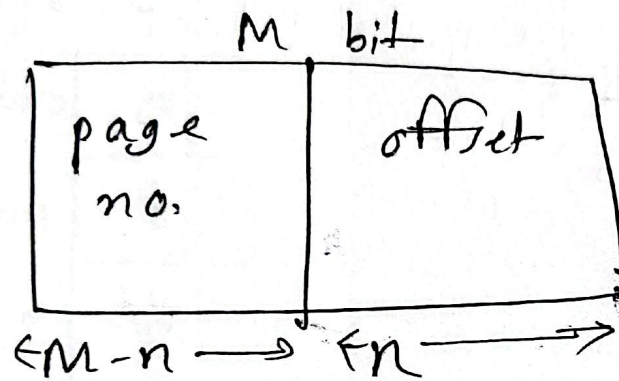
Physical Address = M bits

page size = 2^n

where

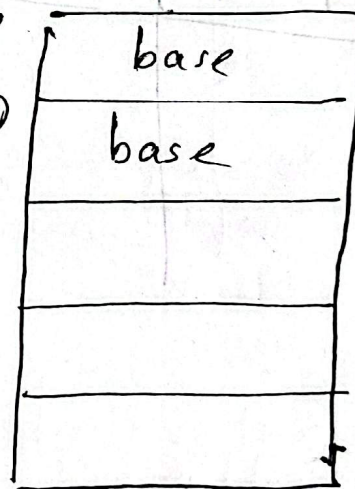
~~n < m~~
 $n < m$

page size have to be something that can be written as 2^n



~~Page tab~~

* Page table:



index / page number
↓

Page table

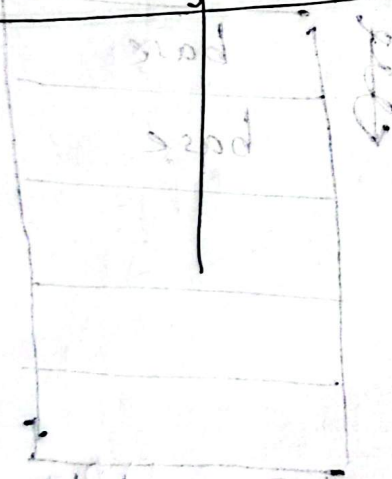
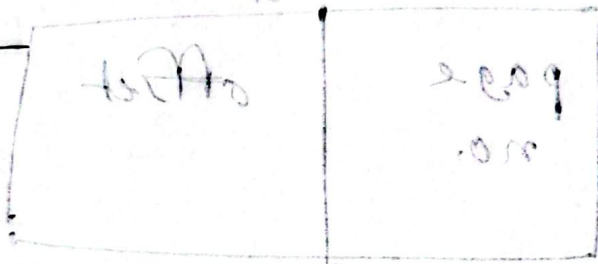
base address = physical memory
frame number

offset

$$PA = \left(\text{frame} \times \frac{\text{frame size}}{\text{page}} \right) + \text{offset}$$

Task-1:

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h



logical page table given
find PM

$$PV = \left(\frac{\text{frame} \times \text{frame size}}{\text{page}} \right) = 19$$

page size = 2^n | Given, $n=2$

$\therefore$ page size = $2^2 = 4$ byte

Page 0

index 0	a
index 1	b
index 2	c
index 3	d

page 0

a, b, c, d four char.

1 char = 1 byte

$\therefore$ 4 char = 4 byte

= page 0

Step - 1: $\text{total no. of frames} = \frac{\text{total memory}}{\text{page size}}$

$$= \frac{32}{4}$$

$$= 8$$

page size = 2
 1. page size = 2

4
 2
 0

12
 16

20

24

28

32

page 0

10	index
8	index
6	index
4	index

212

page 0: page table size: frame = 5

$$\therefore PA = (5 \times 4 + 0)$$

Address

AND 0000 0000 0000 0000
OR 0000 0000 0000 0000
XOR 0000 0000 0000 0000

Flag Affected:

OF = 0
DF = 0
IF = 0

W
R
X

ZF
SF
OF

Reserved

AND: logical multiplication
= bitwise multiplication

Input: 10
Output: 10
AND BL, BL
MUL BL, BL

BL = 1
AL = 2
AH = 0

Input: 0
Output: 0
AND BL, BL
MUL BL, BL

4. exclusion

and wait

5. prevention (No force on resource)

causes

7.3

Resource Allocation graph

2 elemnt
2 elemnt

2.5

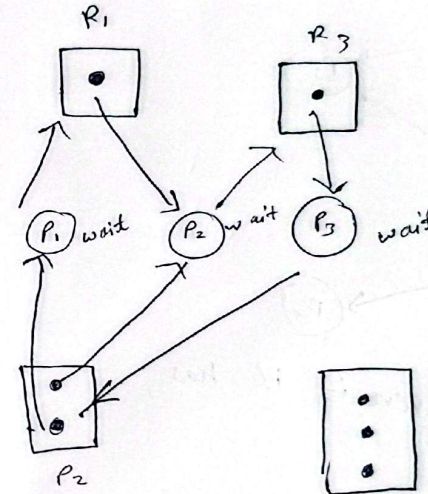
P₁ → R₁

P₁ → R₁

R₁ →



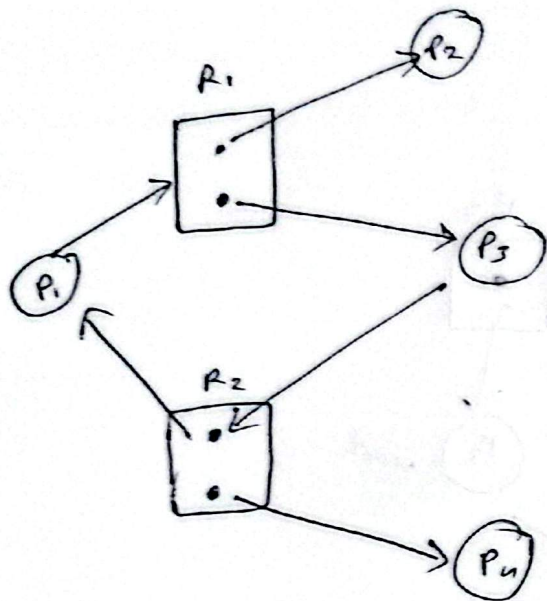
Example



First check if normal circle exists.

All 3 waiting, so, not deadlock.

example



No deadlock, even if it has circular path.

Next class

→ Deadlock

Prevention

Avoidance

Bankers Algorithm.